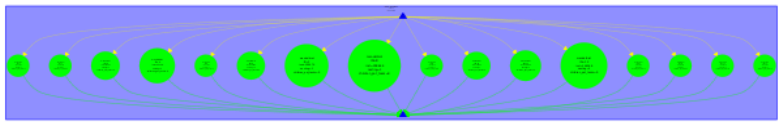
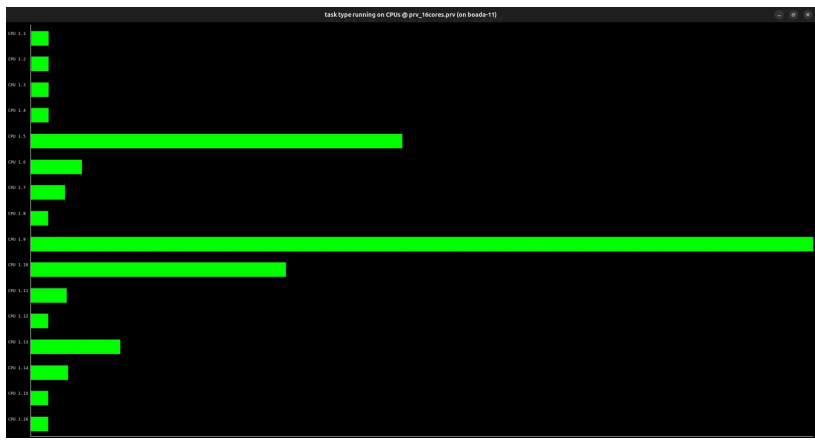


TAREADOR PAR-LAB-3

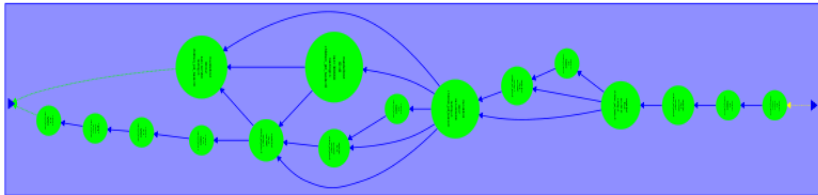
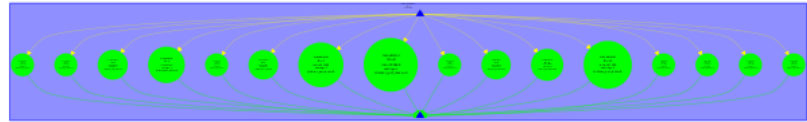
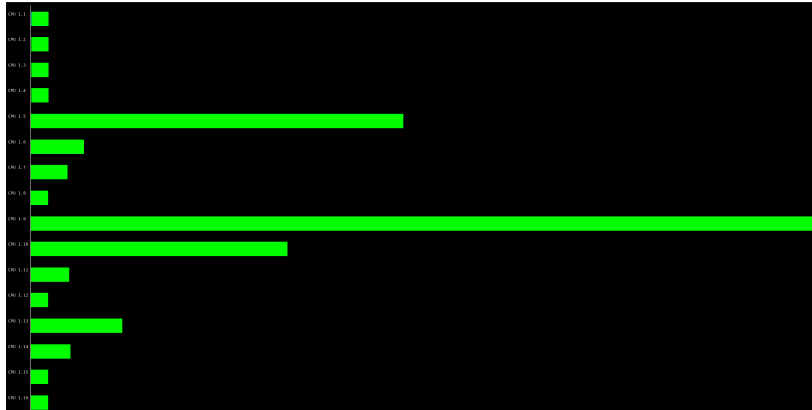
LAB 3.1:

Original tile parallel strategy:

No arguments

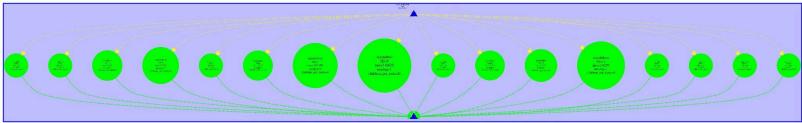
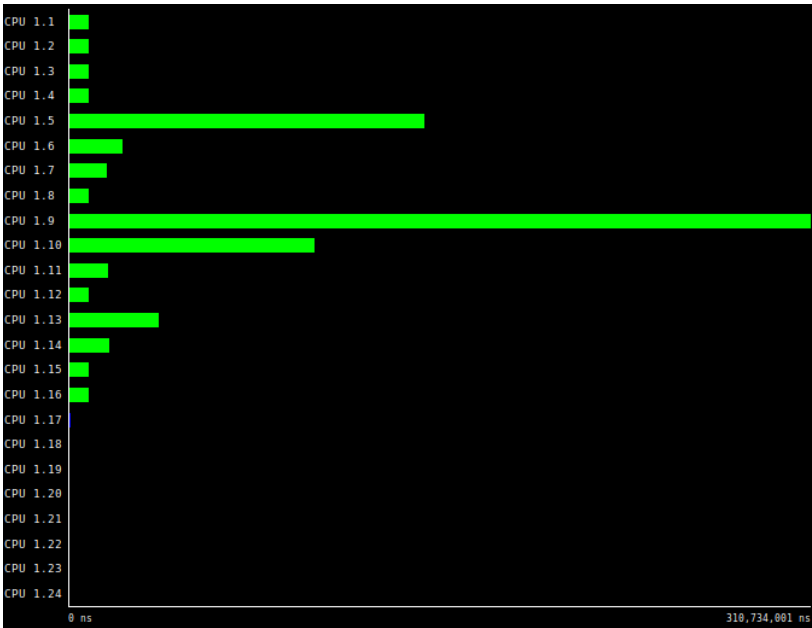
Section	Description
Task decomposition	Original tile
Arguments	None
Code	The code is in mandel-seq-iter-orig-tar.c and we don't modify anything.
Tareador TDG (with dependencies)	
Dependency analysis	Don't have any dependencies, well the MAIN_TAREADOR but other dependencies no.
Tareador TDG (without "data sharing" dependencies)	The same photo as before.
T^∞ analysis	$T_1 = 705$ Mns $T^\infty = 308$ Mns  If we zoom in on the picture we can observe that there is a work imbalance. CPU 9 has a lot more work than the other CPUs. For example CPU 2 is almost not working

With -h

Section	Description
Task decomposition	Original tile
Arguments	-h
Code	The code is in mandel-seq-orig-tar.c and we modify one line of code that's: <code>tareador_disable_object(histogram);</code>
Tareador TDG (with dependencies)	
Dependency analysis	We have one dependency that's "histogram" so we disable it. It's data sharing that we can control with OMP. If we will use that variable we will need to modify the disable function and do it with OMP.
Tareador TDG (without "data sharing" dependencies)	
T^∞ analysis	$T_1 = 713$ Millions ns $T^\infty = 309$ Millions ns  It's very similar to version without arguments If we zoom in on the picture, we can see there is a work imbalance. CPU 9 has a lot more work than the other

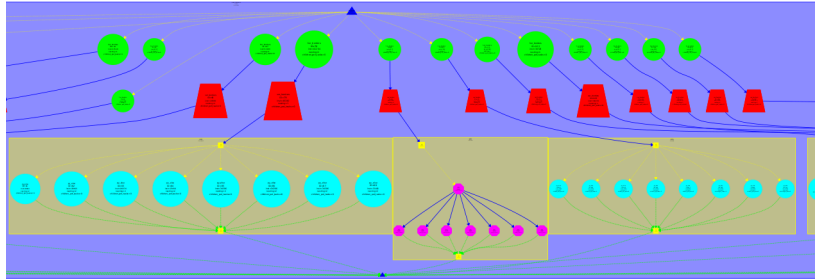
	CPUs. For example, CPU 1 is almost not working.
--	---

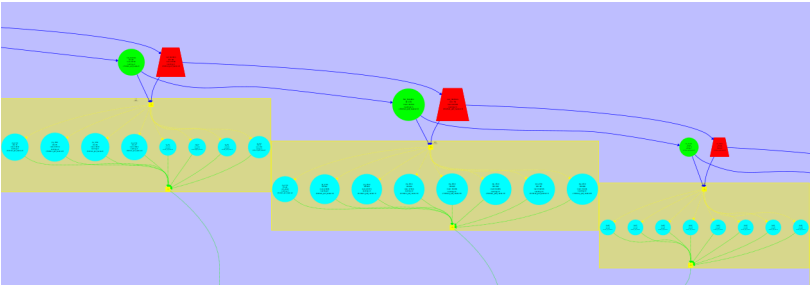
With -d

Section	Description																																																		
Task decomposition	Original tile																																																		
Arguments	-d																																																		
Code	The code is in mandel-seq-iter-orig-tar.c and we modify one line of code that's tareador_disable_object(&X11_COLOR_fake);																																																		
Tareador TDG (with dependencies)																																																			
Dependency analysis	We have one dependency that's "X11_COLOR_fake" so we disable it.																																																		
Tareador TDG (without "data sharing" dependencies)																																																			
T^∞ analysis	T1 = 729 Mns T$^\infty$ = 310 Mns  <table border="1"><caption>Approximate execution times from the bar chart (in ns)</caption><thead><tr><th>CPU</th><th>Approx. Time (ns)</th></tr></thead><tbody><tr><td>1.1</td><td>10,000,000</td></tr><tr><td>1.2</td><td>10,000,000</td></tr><tr><td>1.3</td><td>10,000,000</td></tr><tr><td>1.4</td><td>10,000,000</td></tr><tr><td>1.5</td><td>150,000,000</td></tr><tr><td>1.6</td><td>50,000,000</td></tr><tr><td>1.7</td><td>20,000,000</td></tr><tr><td>1.8</td><td>10,000,000</td></tr><tr><td>1.9</td><td>310,734,001</td></tr><tr><td>1.10</td><td>250,000,000</td></tr><tr><td>1.11</td><td>50,000,000</td></tr><tr><td>1.12</td><td>10,000,000</td></tr><tr><td>1.13</td><td>100,000,000</td></tr><tr><td>1.14</td><td>50,000,000</td></tr><tr><td>1.15</td><td>20,000,000</td></tr><tr><td>1.16</td><td>30,000,000</td></tr><tr><td>1.17</td><td>10,000,000</td></tr><tr><td>1.18</td><td>10,000,000</td></tr><tr><td>1.19</td><td>10,000,000</td></tr><tr><td>1.20</td><td>10,000,000</td></tr><tr><td>1.21</td><td>10,000,000</td></tr><tr><td>1.22</td><td>10,000,000</td></tr><tr><td>1.23</td><td>10,000,000</td></tr><tr><td>1.24</td><td>10,000,000</td></tr></tbody></table> We can see in the TDG that 16 tasks are created by	CPU	Approx. Time (ns)	1.1	10,000,000	1.2	10,000,000	1.3	10,000,000	1.4	10,000,000	1.5	150,000,000	1.6	50,000,000	1.7	20,000,000	1.8	10,000,000	1.9	310,734,001	1.10	250,000,000	1.11	50,000,000	1.12	10,000,000	1.13	100,000,000	1.14	50,000,000	1.15	20,000,000	1.16	30,000,000	1.17	10,000,000	1.18	10,000,000	1.19	10,000,000	1.20	10,000,000	1.21	10,000,000	1.22	10,000,000	1.23	10,000,000	1.24	10,000,000
CPU	Approx. Time (ns)																																																		
1.1	10,000,000																																																		
1.2	10,000,000																																																		
1.3	10,000,000																																																		
1.4	10,000,000																																																		
1.5	150,000,000																																																		
1.6	50,000,000																																																		
1.7	20,000,000																																																		
1.8	10,000,000																																																		
1.9	310,734,001																																																		
1.10	250,000,000																																																		
1.11	50,000,000																																																		
1.12	10,000,000																																																		
1.13	100,000,000																																																		
1.14	50,000,000																																																		
1.15	20,000,000																																																		
1.16	30,000,000																																																		
1.17	10,000,000																																																		
1.18	10,000,000																																																		
1.19	10,000,000																																																		
1.20	10,000,000																																																		
1.21	10,000,000																																																		
1.22	10,000,000																																																		
1.23	10,000,000																																																		
1.24	10,000,000																																																		

	Tareador. So if we use 24 CPUs/threads we can see that there is a lot of work unbalance at the middle/end of the execution, just at the beginning parallelisation is useful.
--	--

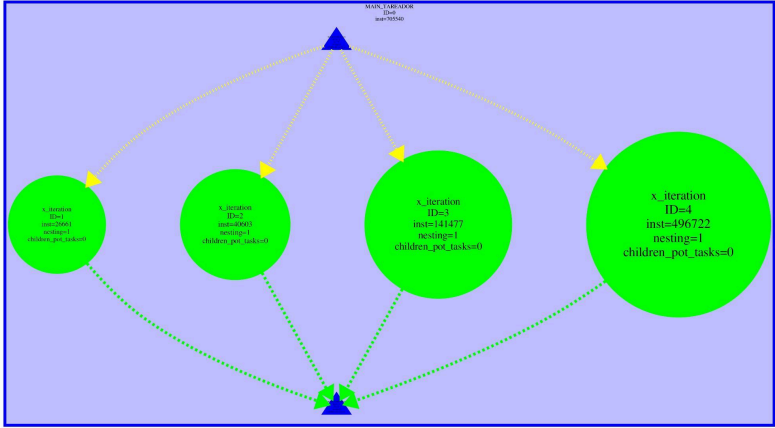
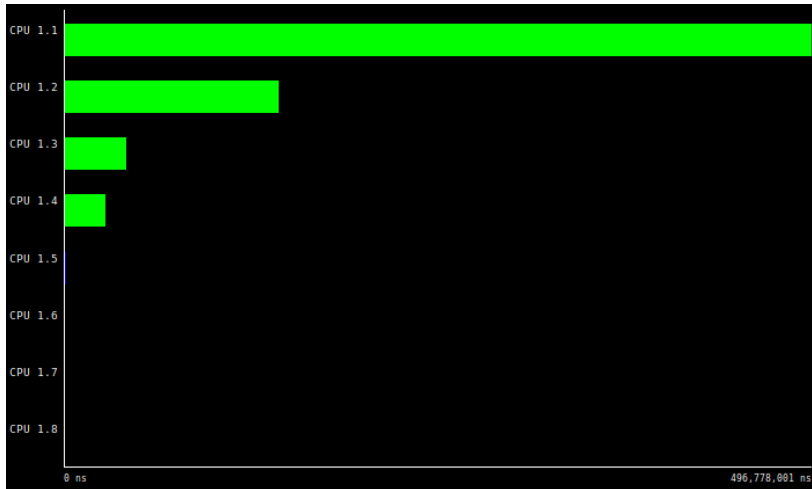
Fine grain parallel strategy:

Section	Description
Task decomposition	Fine grain parallel strategy
Arguments	None
Code	The code is in mandel-seq-iter-fine-tar.c and the important parts is that: <pre> equalh = 1 / equalv = 1 and equal = equalh & equalv; tareador_disable_object(&X11_COLOR_fake); tareador_disable_object(histogram); tareador_disable_object(Hmatrix); </pre>
Tareador TDG (with dependencies)	
Dependency analysis	We have two big problems with dependencies, which are: <pre> tareador_disable_object(&X11_COLOR_fake); tareador_disable_object(histogram); </pre> These two dependencies cause very poor parallelism in the program, so we disabled them. We also observed that we could disable the equal dependency, but for this, we needed to split the variable equal into equalh and equalv. This is because the if(equal) statement cannot run in parallel with the first two loops if they share the same variable. The variable Hmatrix it's data sharing. We can disable it because we don't use it in the future, if we had to use it we would need to use some OMP features like critical or atomic.

Tareador TDG (without “data sharing” dependencies)	
T [∞] analysis	T1 = 740 Mns T[∞] = 80 Mns  Here we can still see some unbalanced tasks. Some CPUs, such as 2, 15, and 16, finish early and don't continue, while others keep working with long tasks like cyan task until the end of the execution

Column of tiles parallel strategy:

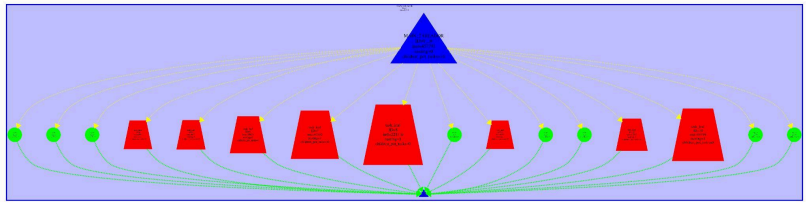
Section	<u>Description</u>
Task decomposition	Column of tiles parallel strategy
Arguments	None
Code	The code is in mandel-seq-col-tar.c and the important part is the main loops: <pre> for (int x = 0; x < COLS; x += TILE) { tareador_start_task("x_iteration"); for (int y = 0; y < ROWS; y += TILE) { ... } tareador_end_task("x_iteration"); } </pre>

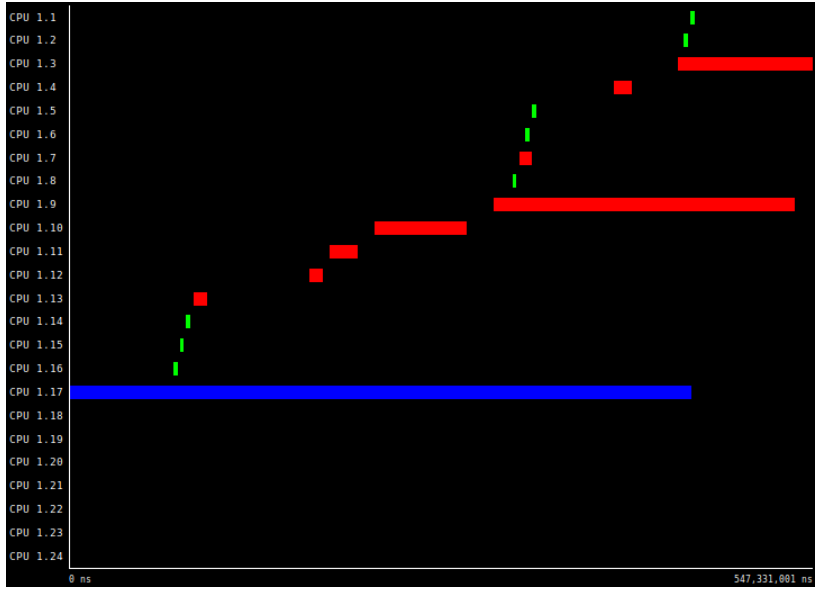
Tareador TDG (with dependencies)	
Dependency analysis	We have only declared one task according to the instructions, and it's created at every iteration of the x loop. By default COLS = 32 and TILE = 8, and that makes 4 iterations, the same as tasks created. Because each task is an iteration, each task will have an x with its own value, making the parallelisation plausible and therefore, no dependencies between tasks.
Tareador TDG (without “data sharing” dependencies)	As said before, there are no dependencies that interrupt the parallelism, so the TDG would be the same as the previous one.
T [∞] analysis	T₁ = 705 Mns T[∞] = 408 Mns 

	With this strategy 4 tasks are created. If we use 8 CPUs/threads we can see that there is some load unbalance, but not that much.
--	---

LAB 3.2

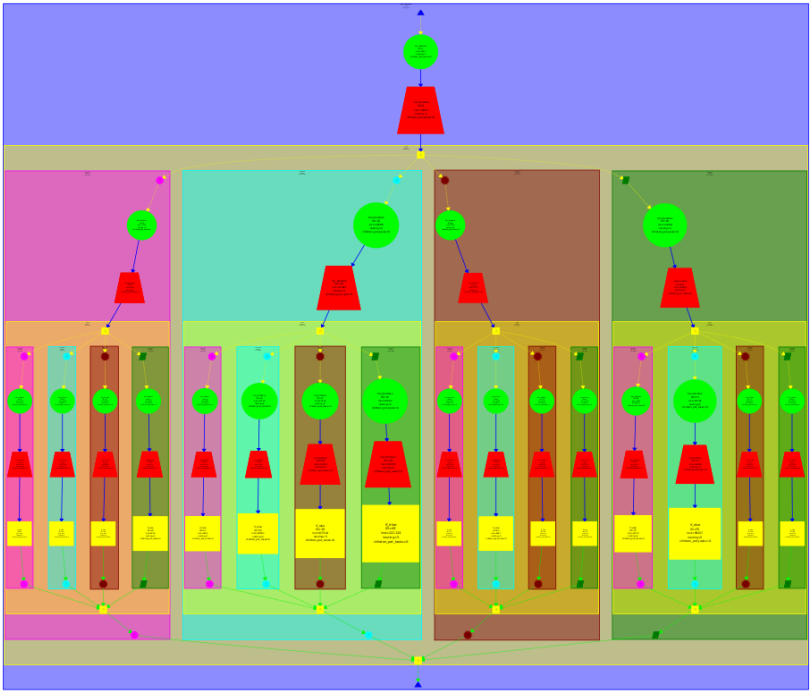
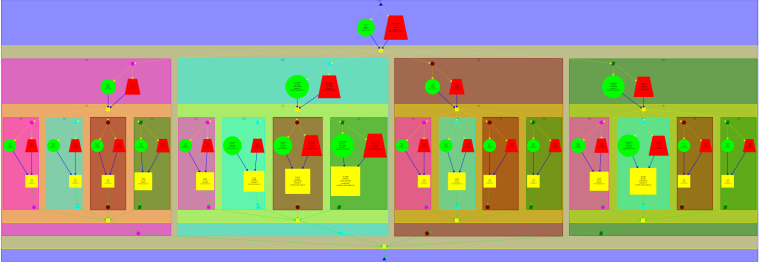
Leaf recursive parallel strategy:

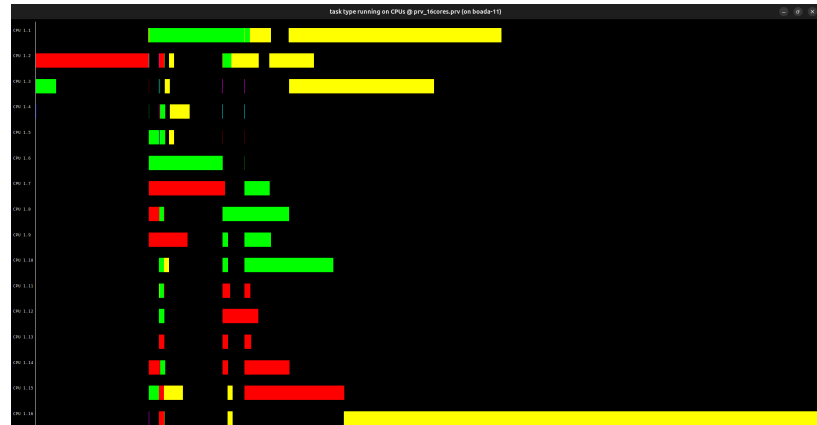
Section	Description
Task decomposition	Leaf recursive parallel strategy
Arguments	None
Code	<pre> if (equal) { // fill all with the same value: base case tareador_start_task("task_equal"); tareador_end_task("task_equal"); } else { if (NCols <= TILE) { // computation of tile: base case tareador_start_task("task_leaf"); tareador_end_task("task_leaf"); } } </pre>
Tareador TDG (with dependencies)	
Dependency analysis	As we can see in the TDG, the yellow arrows are for control variables, so there are not any dependencies between tasks.
Tareador TDG (without “data sharing” dependencies)	As said before, there are no dependencies that interrupt the parallelism, so the TDG would be the same as the previous one.

T^∞ analysis	$T_1 = 922 \text{ Mns}$ $T^\infty = 547 \text{ Mns}$  We have seen that with this strategy 16 tasks are created. Hence, if we use 24 CPU/threads we can see that some work unbalance happens, some CPUs work a lot whether some do almost 0 work.
---------------------	--

Tree Strategy

Section	<u>Description</u>
Task decomposition	Tree recursive parallel strategy
Arguments	None
Code	The code is in mandel-seq-rec-tar-tree.c and we modify the code in many section, but we're only putting tasks, the tasks are the same horizontal, vertical and if/else task, also we code tasks in every recursive call

Tareador TDG (with dependencies)	
Dependency analysis	We have one dependency on the equal variable, we don't disable it because this variable we use it later, so to remove this dependency we split the equal in equalh and equalv and later we merge in one variable named equal.
Tareador TDG (without "data sharing" dependencies)	
T^∞ analysis	$T_1 = 922$ Mns $T^\infty = 364$ Mns



Looking at this simulation, there is a lot of concurrency among the tasks. However, we can see an unbalanced task when finishing on CPU 16. If we try with 24 CPUs, we can see more or less the same parallelism, but the tasks are more spread out.



Summary Table

Task Decomposition	Strategy	Run Arguments	T1	T ∞	Parallelism	Load unbalance: yes/no, why?
Iterative	Original	None	705 Mns	308 Mns	2,29	Yes. Some tasks like calculating the outside points take much more time. Those that finish much before then do nothing.
		-d	729 Mns	310 Mns	2,35	Yes. Identical as before.
		-h	713 Mns	309 Mns	2,31	Yes. Identical as before.
	Fine Grain	None	740 Mns	80 Mns	9,25	Yes. But it seems that there is not as much as before.
	Column	None	705 Mns	408 Mns	1,73	Yes. Same as previous cases.
Recursive	Leaf	None	922 Mns	547 Mns	1,69	Yes. Happens the same.
	Tree	None	922 Mns	364 Mns	2,53	Yes. But not as much as before.
Best Task Decomposition	Strategy	Reason Why				
	Fine Grain	It gives the most parallelism among all the options, and does not provide as much load unbalance as the others.				